



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato finale in **Networks and Cloud Infrastructures**

***L4AntiDos: Monitoraggio Dinamico e
Mitigazione di Attacchi TCP/UDP/ICMP
Flooding In Reti SDN***

Anno Accademico 2025/2026

Membri:

Valentino Alberobello

Angela Dalia

Daniele Di Sarno

1. Introduzione e obiettivi del progetto

Nel panorama attuale delle infrastrutture di rete, la tecnologia **Software-Defined Networking (SDN)** rappresenta un modello molto importante, perché separa il **piano di controllo** dal **piano dei dati**. Questa struttura rende la rete più flessibile e programmabile, ma allo stesso tempo espone il controller a possibili attacchi, soprattutto di tipo **Denial of Service (DoS)**. In particolare, attacchi basati su protocolli di livello 3 e 4, come ICMP, TCP e UDP, possono saturare i collegamenti di rete e aumentare il carico sugli switch e sul controller. L'obiettivo di questo progetto è stato realizzare e testare un meccanismo di difesa automatico basato su **Ryu Controller** e **OpenFlow v1.3**. L'applicazione sviluppata, denominata **L4AntiDoS**, monitora i flussi di traffico presenti nella rete e calcola soglie dinamiche in base alla larghezza di banda reale delle porte configurate nella topologia Mininet. Le informazioni relative alla larghezza di banda delle porte vengono salvate dallo script della topologia in un file JSON, che viene successivamente letto dal controller per calcolare le soglie di traffico. Il sistema è stato verificato attraverso diversi test, sia con traffico regolare sia con traffico anomalo generato tramite **D-ITG**, che permette di personalizzare protocollo, destinazione, dimensione dei pacchetti e frequenza di invio. Quando il traffico supera la soglia prevista, il controller identifica l'host attaccante e applica un **ban globale** tramite regole OpenFlow di drop. È stata inoltre testata una variante con **unban temporaneo**, che rimuove automaticamente il blocco dopo un intervallo prestabilito. Infine, il funzionamento del controller è stato verificato anche su una topologia alternativa composta da **4 host e 3 switch**, confermando che il sistema non dipende da una sola configurazione di rete, ma è in grado di adattare le soglie in modo dinamico anche in scenari diversi.

1.1 Ambiente di emulazione e strumenti di sviluppo

Per l'ideazione, la scrittura del codice e la successiva fase di testing sperimentale, è stato configurato un ambiente di sviluppo basato sull'interazione di diversi strumenti software:

- **Oracle VM VirtualBox:** Utilizzato come hypervisor per l'esecuzione di una macchina virtuale Linux (Ubuntu Server).
- **Mininet:** Il core dell'ambiente di emulazione. Eseguito interamente da riga di comando (CLI) all'interno della macchina virtuale, consente di creare ed emulare la topologia di rete.
- **Tshark:** La versione a riga di comando di Wireshark, eseguita all'interno della macchina virtuale per l'analisi e la cattura dei pacchetti in tempo reale.
- **Visual Studio Code (VS Code):** Utilizzato come ambiente di sviluppo integrato (IDE) sul sistema operativo host.
- **Estensione Remote - SSH:** Tramite questa estensione, VS Code stabilisce una sessione SSH sicura verso la macchina virtuale VirtualBox. Questo ha permesso di modificare i file di configurazione della topologia e la logica del controller Ryu direttamente dall'IDE dell'host.

2. Architettura di rete e topologia

L'architettura è composta da un controllore (Ryu Controller) collegato a due switch logici, denominati **Switch 1 (S1)** e **Switch 2 (S2)**.

La configurazione dei nodi terminali (host) è la seguente:

- **Host 1 (H1) - IP 10.0.0.1**
- **Host 2 (H2) - IP 10.0.0.2**
- **Host 3 (H3) - IP 10.0.0.3**

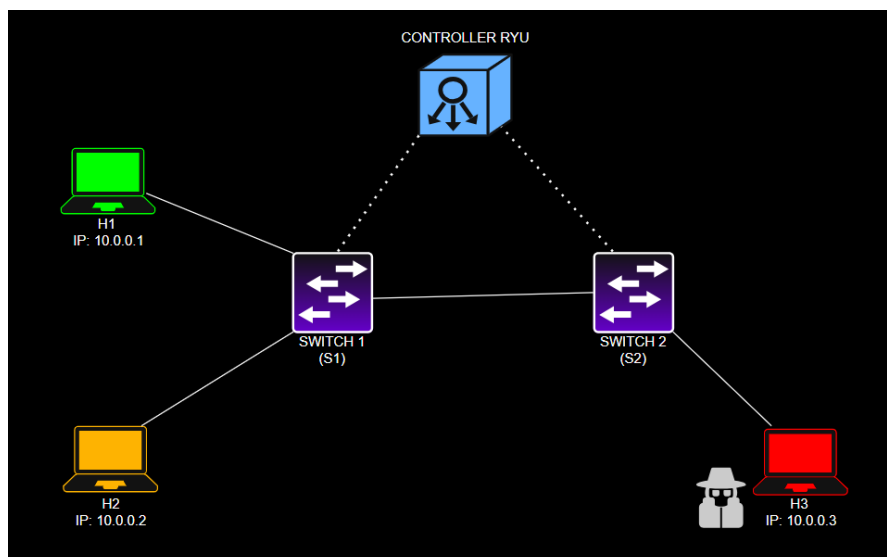


Figura 1 - Topologia di Rete

3. Caratteristiche generali

Una delle caratteristiche principali del sistema è l'adozione di un approccio dinamico per la definizione delle soglie di anomalia dei flussi di pacchetti. Invece di configurare un valore fisso di pacchetti al secondo valido per tutta la rete, l'applicazione rileva i vincoli di banda ad ogni porta e adatta la tolleranza di conseguenza.

All'avvio dell'applicazione, il controller tenta di leggere un file di configurazione in formato JSON posizionato in /tmp/port_bw.json. Questo file viene popolato dallo script della topologia mininet e mappa in modo univoco ogni coppia (**DPID**, **Port_No**) con la rispettiva larghezza di banda nominale espressa in Mbps. Nel caso in cui il file non sia presente, viene adottato un valore di fallback pari a 10 Mbps, questo solo per non rendere l'esecuzione del codice incline ai crash ma di fatto ne influisce il funzionamento perché non risulta più quello atteso.

Il calcolo della soglia massima di pacchetti consentiti per ciascun intervallo di monitoraggio avviene tramite l'applicazione sequenziale delle seguenti formule logiche:

$$BW_bps = BWMbps \times 1.000.000$$

Una volta determinata la capacità totale del collegamento, stabiliamo che una singola tipologia di traffico (come, ad esempio, le richieste ICMP Echo) non possa occupare più del 3% della banda disponibile. Nel codice, questo limite è definito dal parametro **Threshold_Ratio** = 0.03. Assumendo una dimensione media del pacchetto di rete pari a **BitsPerPkt** = 12000 bit (corrispondenti a 1500 Byte), il tasso di pacchetti teorico al secondo si ricava come:

$$Pkts_Per_Sec = \frac{BWbps \times ThresholdRatio}{BitsPerPkt}$$

Dato che il thread di monitoraggio interroga gli switch ad intervalli regolari definiti da **Monitor_Interval** = 0.5 secondi, la soglia discreta del link (Threshold) ai fini del ban dell'host che sta effettuando l'attacco DoS, è uguale a:

$$Threshold = Pkts_Per_Sec \times Monitor_Interval$$

Banda del Link (Mbps)	Pacchetti al Secondo	Threshold (Ogni 0.5s)
10 Mbps (Fallback)	25 pkts/s	12,5 pkts/s
100 Mbps	250 pkts/s	125 pkts/s
1'000 Mbps	2'500 pkts/s	1'250 pkts/s
10'000 Mbps	25'000 pkts/s	12'500 pkts/s

4. Funzionamento – comandi e output

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Figura 2 – Comando Pingall

L'esecuzione del comando **pingall** certifica la completa raggiungibilità bidirezionale tra tutti i nodi della rete, attestando il corretto instradamento dei pacchetti con un tasso di perdita nullo (0% dropped, 6/6 pacchetti ricevuti).

```
*** Starting CLI:
mininet> h3 ping -c 4 h1
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=17.3 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.15 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=0.190 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.730 ms
--- 10.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3014ms
rtt min/avg/max/mdev = 0.190/4.838/17.289/7.196 ms

instantiating app ryu.controller.ofp_handler of OFPHandler
[*] Anti-DoS attivo su Switch ID: 1
[*] Anti-DoS attivo su Switch ID: 2
[PORTA] dpid=1 porta=1 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[PORTA] dpid=1 porta=2 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[PORTA] dpid=1 porta=3 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[PORTA] dpid=2 porta=1 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[PORTA] dpid=2 porta=2 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[OK] 82:42:a2.. porta=2 rate=1 pkts/0.5s - sotto soglia (1250)
[OK] 82:42:a2.. porta=2 rate=1 pkts/0.5s - sotto soglia (1250)
[OK] 82:42:a2.. porta=2 rate=1 pkts/0.5s - sotto soglia (1250)
```

Figura 3 – Invio controllato di pacchetti ICMP da h3 ad h1 (ban non rilevato)

A sinistra, tramite il terminale di Mininet, viene avviato il comando **h3 ping -c 4 h1**. Il risultato conferma che i 4 pacchetti ICMP vengono trasmessi e ricevuti con successo (0% di pacchetti persi), dimostrando che la comunicazione tra gli host è attiva e stabile.

A destra, I messaggi [OK] confermano che il traffico ICMP viaggia ad un rate di un solo pacchetto ogni mezzo secondo, rimanendo ampiamente al di sotto del threshold (1250 pkts/0.5s). Di conseguenza, il controller non genera alcun alert e non applica alcuna regola.

```
mininet> h2 ITGRecv &
mininet> h3 tshark -i h3-eth0 -w /tmp/cattura_mininet.pcap -F pcap &
mininet> h3 ITGSend -a 10.0.0.2 -T UDP -c 1200 -C 5000
Running as user "root" and group "root". This could be dangerous.
Capturing on 'h3-eth0'
** (tshark:4955) 20:53:44.409539 [Main MESSAGE] -- Capture started.
** (tshark:4955) 20:53:44.409622 [Main MESSAGE] -- File: "/tmp/cattura_mininet.pcap"
10 ITGSend version 2.8.1 (r1023)
Compile-time options: sctp dccp bursty multiport
Started sending packets of flow ID: 1

[PORTA] dpid=2 porta=2 -> 1000 Mbps -> soglia=1250 pkts/0.5s
[OK] 42:7b:94.. porta=2 rate=4 pkts/0.5s - sotto soglia (125)
[OK] 62:fd:aa.. porta=2 rate=34 pkts/0.5s - sotto soglia (1250)

=====
[ALERT] DoS rilevato su switch 2
Attaccante : 62:fd:aa:b6:1f:62
Vittima : 42:7b:94:f5:4e:e7
Rate : 1912 pkts/0.5s > soglia 1250 pkts/0.5s
Azione : BAN applicato su tutta la rete
```

Figura 4 - Generazione Traffico UDP con DITG (scenario BAN)

La figura mostra l'avvio della simulazione tramite **D-ITG**, uno strumento che permette di personalizzare il traffico generato. In questo modo è possibile simulare un attacco UDP controllato e verificare la risposta del sistema Anti-DoS. Su **h2** viene eseguito **ITGRecv**, così l'host resta in ascolto, mentre su **h3** viene avviata la cattura dei pacchetti con tshark sull'interfaccia h3-eth0. Successivamente viene eseguito il comando **ITGSend -a 10.0.0.2 -T UDP -c 1200 -C 5000**, con cui **h3** genera traffico UDP verso **h2**. In questo caso i pacchetti hanno una dimensione di 1200 byte, scelta per ottenere una cattura più ordinata e senza frammentazione.

Nel terminale del controller Ryu si vede poi il rilevamento dell'anomalia: il traffico raggiunge un rate di **1912 pkts/0.5s**, superiore alla soglia configurata di **1250 pkts/0.5s**. Il controller identifica quindi **h3** come attaccante e **h2** come vittima, applicando automaticamente il **BAN su tutta la rete** tramite regole OpenFlow.

```

mininet@mininet-vm:~$ sudo tshark -r /tmp/cattura_mininet.pcap -Y "udp" | head -n 10
Running as user "root" and group "root". This could be dangerous.
 13  0.038193  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 15  0.039307  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 17  0.043387  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 19  0.044192  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 21  0.045361  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 23  0.046114  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 25  0.046683  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 27  0.047380  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 29  0.049116  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500
 31  0.049657  10.0.0.3 → 10.0.0.2    UDP 62 39858 → 8999 Len=1500

```

Figura 5 – Cattura Pacchetti Tramite TShark

La figura mostra la lettura dei primi 10 pacchetti UDP catturati nel file /tmp/cattura_mininet.pcap. Tutti i pacchetti risultano inviati da **10.0.0.3** verso **10.0.0.2**, confermando il traffico generato durante la simulazione con **D-ITG**. L'aspetto più importante è il campo **Time**: il primo pacchetto viene registrato al tempo **0.038193 s**, mentre il decimo al tempo **0.049657 s**. La differenza è quindi: $\Delta t = 0.049657 - 0.038193 = 0.011464 \text{ s}$, cioè circa **11,46 ms**. Questo significa che vengono inviati **10 pacchetti in poco più di 11 millisecondi**, mostrando un flusso UDP molto ravvicinato nel tempo. Tale comportamento conferma l'intensità del traffico generato e giustifica il successivo rilevamento dell'anomalia da parte del controller Anti-DoS.

```

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 X
h2 -> h1 X
h3 -> X X
*** Results: 66% dropped (2/6 received)
mininet>

```

Figura 6 - Pingall dopo Ban H3

La figura 6 mostra l'esecuzione del comando pingall dopo il rilevamento dell'attacco DoS e l'applicazione del ban da parte del controller Ryu. Il risultato evidenzia che **h3** risulta isolato dalla rete: infatti non riesce più a comunicare né con **h1** né con **h2**. Al contrario, **h1** e **h2** continuano a raggiungersi correttamente tra loro. Il valore finale, **66% dropped**, conferma che una parte consistente dei pacchetti viene bloccata perché l'host attaccante è stato escluso. Questo dimostra che il ban globale applicato tramite regole OpenFlow funziona correttamente, bloccando h3 senza interrompere la comunicazione tra gli host legittimi.

5.Approfondimenti: Unban temporaneo e Topologia Alternativa

5.1 Variante Ryu-Controller con Meccanismo di Un-BAN

Al fine di evitare l'isolamento permanente degli host, è stata implementata una variante con *unban* automatico dopo 60 secondi. Il meccanismo riduce gli effetti dei falsi positivi, impedendo che un utente legittimo resti bloccato indefinitamente. Nel codice effettivo il timer è gestito dal controller Ryu: le regole DROP vengono installate senza scadenza automatica e una routine asincrona attende 60 secondi prima di ordinarne la cancellazione agli switch. Il funzionamento coinvolge i due piani nel modo seguente:

1. **Data Plane:** sugli switch vengono installate due regole *DROP* ad alta priorità, una sul MAC sorgente e una sul MAC destinazione dell'attaccante.
2. **Control Plane:** `hub.spawn()` avvia in background `_unban_host()`. Dopo `hub.sleep(60)`, la funzione invia ad ogni switch due FlowMod con comando `OFPF_DELETE`, rimuove il MAC da `already_blocked` e cancella da `flow_history` le informazioni collegate all'host.

Analisi dei Trade-off: Sebbene questa variante aumenti la resilienza e l'autocorrezione della rete, essa introduce una vulnerabilità al fenomeno del flapping qualora l'attacco DoS sia automatizzato e continuo. In tale scenario, alla scadenza della regola sullo switch, il traffico malevolo torna a colpire direttamente il Control Plane tramite messaggi Packet-In, generando picchi periodici di carico sul controller.

```
=====
[ALERT] DoS rilevato su switch 1
Attaccante : f6:4d:7e:58:22:f4
Vittima    : 0a:80:96:7a:79:58
Rate       : 202 pkts/0.5s > soglia 125 pkts/0.5s
Azione     : BAN applicato su tutta la rete
=====

[*] [UNBAN] f6:4d:7e:58:22:f4 sbloccato, regole DROP rimosse.
□
```

Figura 7 – Rilevamento DoS e sblocco automatico dell'host

La figura 7 mostra il comportamento della variante con *unban temporaneo*. Il controller Ryu rileva un attacco DoS sullo **switch 1**, identifica l'host attaccante e applica il **BAN su tutta la rete** mediante regole OpenFlow di DROP. Dopo 60 secondi, `_unban_host()` invia agli switch i comandi di cancellazione e aggiorna le strutture dati interne. L'host viene così sbloccato senza utilizzare un timeout automatico delle regole OpenFlow.

5.2 Variante di topologia: verifica della dinamicità del controller Ryu

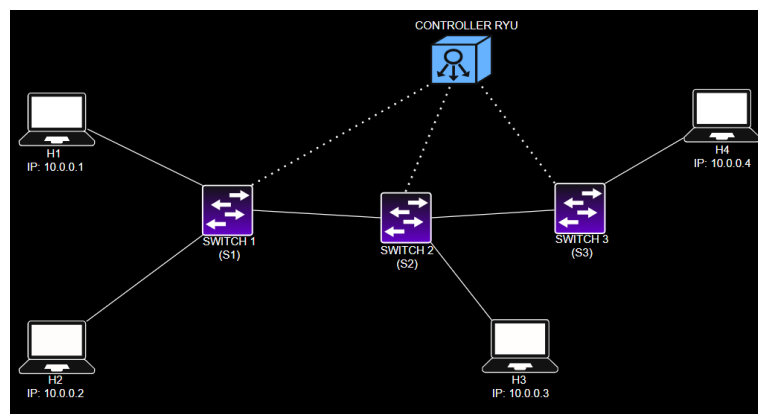


Figura 8 – Topologia alternativa per la verifica del controller

Al fine di validarne la flessibilità, il controller Ryu è stato testato su una seconda topologia, più estesa e complessa rispetto a quella principale. Questo scenario, caratterizzato da tre switch, quattro host e collegamenti con larghezze di banda eterogenee ha permesso di dimostrare l'assoluta dinamicità del sistema. Il vero punto di forza di questa architettura risiede nel fatto che **il codice del controller Ryu rimane rigorosamente invariato ed estraneo a qualsiasi modifica** tra le esecuzioni di topologie differenti. L'adattamento ai mutamenti della rete avviene infatti in modo del tutto autonomo: il controller riceve le informazioni sulle bande e calcola le relative soglie basandosi sul file `/tmp/port_bw.json`. È fondamentale sottolineare che **questo file non richiede alcun intervento o configurazione manuale da parte nostra**; viene infatti generato e popolato automaticamente dallo script Python di Mininet all'avvio della topologia stessa.

Sebbene i dati specifici non siano stati integrati nel testo per ragioni di sintesi, i test di rilevamento e mitigazione eseguiti su questa rete estesa hanno confermato la perfetta reattività del sistema Anti-DoS e l'efficacia del meccanismo di ban, comprovando la nativa capacità del controller di operare in ambienti di rete differenti senza toccare una singola riga di codice.